

PREMIUM BLUEPRINT · LEARN → BUILD → PRESENT

Crypto Pulse

A Guided Summer Path into Data Engineering
Modern Data Stack · Internship-Ready Portfolio · Adaptable Bachelor Capstone

- Docker Compose
- Python
- MinIO / S3
- Apache Parquet
- PostgreSQL
- DuckDB
- dbt Core
- Prefect / Airflow
- Metabase
- GitHub

Two tracks, one build

This document works as (1) a self-paced **learning curriculum** that turns you from "knows Python" into "can build a production-style data platform", and (2) a **portfolio + bachelor capstone package** — the engineering work is identical, and Section 9 shows exactly how to re-frame the same repository as a formal thesis.

- Time budget:** 10-11 weeks · 5-8 hrs/week · ~60-85 hours total
- Target roles:** Data Engineer / Analytics Engineer Intern, Junior Data Platform Engineer
- Academic use:** Suitable as a Bachelor's capstone/thesis project in Computer Engineering (see Section 9)
- Prerequisites:** Basic Python syntax, basic SQL (SELECT/JOIN/GROUP BY), comfort typing commands in a terminal — nothing more is assumed

0 · How to Use This Document

Every phase below follows the same loop: **Learn** the minimum concept set needed → **Build** the corresponding piece of the pipeline immediately while it's fresh → **Check** yourself against a Definition of Done and a few diagnostic questions before moving on. Don't binge all the learning material up front — you retain almost nothing that way. Learn just enough for the week's build task, then build.

Realistic pacing: 10–11 weeks × 5–8 hrs/week ≈ 60–85 hours. That is enough to build a genuinely working, well-tested MVP of every phase below — it is *not* enough to gold-plate every feature. If you fall behind, protect the "Definition of Done" checklist for each phase over extra polish; a complete, working Phase 3 beats a half-finished Phase 3 and a beautiful Phase 5 you never reach.

Data source note: This plan defaults to the **CoinGecko free Demo API** (register a free key, no credit card required) as the primary source — its free tier gives a stable 100 calls/minute and 10,000 calls/month, which is comfortable for polling 15–20 coins every few minutes. Binance's public REST API is offered as an optional secondary source in Phase 1 for extra ingestion-resilience practice (handling two schemas); check current regional API availability before relying on it, as exchange API access can vary by jurisdiction.

Table of Contents

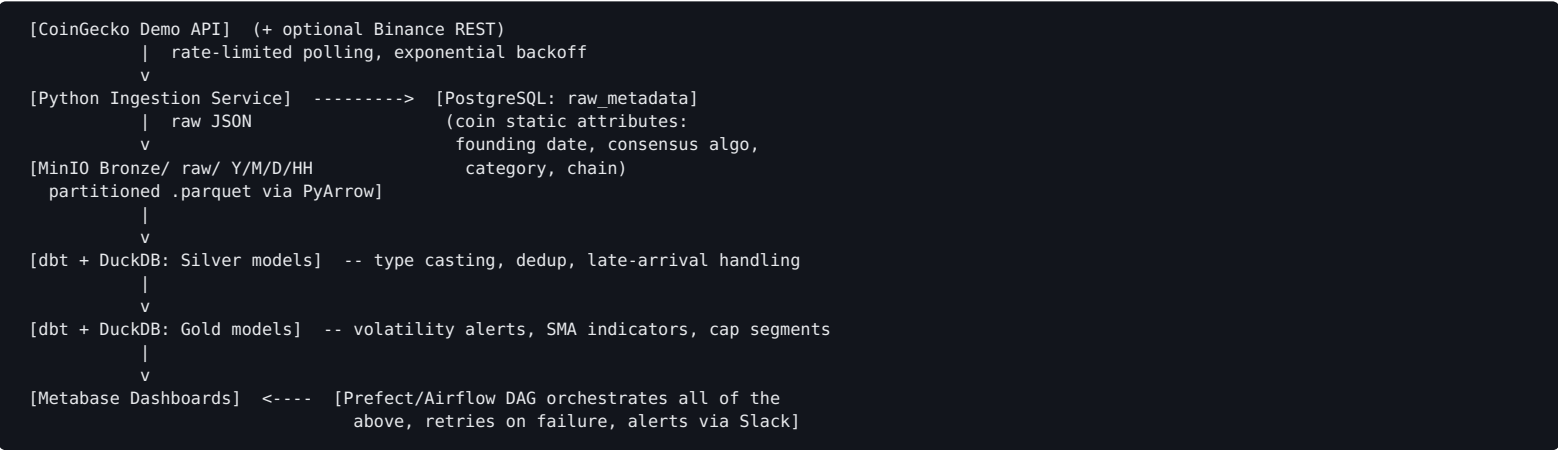
1 · Executive Summary	2
2 · System Architecture	2
3 · Data Model & Business Scenario	3
4 · Repository Structure	3
5 · Weekly Learn → Build Plan (Phases 1–5)	4
6 · Learning Resource Library	9
7 · Skills-to-Job-Posting Alignment	10
8 · Risks & Mitigations	10
9 · Bachelor Capstone Appendix (Academic Track)	11
10 · Overall Success Metrics & Final Checklist	13

1 • Executive Summary

Crypto Pulse is an end-to-end data platform that ingests live cryptocurrency market data, lands it in a versioned data lake, transforms it through a governed Medallion Architecture, and serves the result through an orchestrated, tested, monitored BI layer. It mirrors the toolchain most frequently requested in current Data Engineering and Analytics Engineering job postings, so every phase produces both a working system component *and* a specific interview or thesis talking point.

What makes this "portfolio-grade" rather than a tutorial project: resilience (retries, backoff, idempotent loads), observability (structured logs, failure alerting), data contracts (dbt tests + schema enforcement), and reproducibility (one command spins up the entire stack from empty volumes).

2 • System Architecture



Layer	Technology	Rationale
Containerization	Docker & Docker Compose	Reproducible local environment; appears in the large majority of current DE listings.
Ingestion	Python	Rate limits, pagination, retries, defensive JSON parsing — the most commonly tested "real-world" DE skill.
Data Lake	MinIO (S3-compatible) + Parquet	Mirrors AWS S3 in interviews without cloud cost; Parquet is the de-facto lake standard.
Warehouse / Compute	PostgreSQL + DuckDB	Postgres models slowly-changing metadata; DuckDB gives fast local OLAP over Parquet.
Transformation	dbt Core	Testable, documented, version-controlled SQL transformation logic.
Orchestration	Prefect (recommended) or Airflow	Converts scripts into monitored, retryable, schedulable DAGs.
BI	Metabase	Open-source, fast to stand up, shareable dashboards for a demo.

3 • Data Model & Business Scenario

Scenario: simulate a trading-desk analytics function blending real-time market data with static reference data to power alerting and portfolio segmentation.

Dynamic (API, polled every 1-5 min)

- Symbol, price (USD), 24h volume
- 24h % change, market cap
- Timestamp (ingestion + exchange)

Static (PostgreSQL seed / slowly changing)

- Coin founding date, chain / consensus algorithm
- Category (Layer 1, DeFi, stablecoin, meme, etc.)
- Circulating vs. max supply

Gold-Layer Star Schema

Table	Grain	Key Columns
fct_price_snapshot	1 row / coin / interval	coin_id, event_ts, price_usd, volume_24h, pct_change_24h
dim_coin	1 row / coin (SCD Type 2)	coin_id, symbol, category, consensus_algo, founded_date, valid_from, valid_to
fct_volatility_alerts	1 row / breach event	coin_id, event_ts, pct_move, direction, threshold_breached
fct_technical_indicators	1 row / coin / day	coin_id, date, sma_7, sma_25, signal

4 • Repository Structure

```
crypto-pulse/
├─ docker-compose.yml
├─ .env.example
├─ ingestion/{extract_prices.py, extract_metadata.py, clients/}
├─ lake/json_to_parquet.py
├─ warehouse/init_postgres.sql
├─ transform/dbt_project/{models/bronze,silver,gold, tests/}
├─ orchestration/flows/crypto_pulse_flow.py
├─ dashboards/metabase_exports/
├─ docs/architecture.png
├─ benchmarks/parquet_vs_json.md
└─ README.md
```

5 · Weekly Learn → Build Plan

11 weeks across 5 phases. Each week lists what to learn first, what to build immediately after, and how to know you're actually done. Full resource links are in Section 6 — names in *italics* below map to that table.

Phase 1 · Infrastructure & Resilient Ingestion WEEKS 1-2				
Goal: an isolated, reproducible environment and raw extraction that survives real API failures.				
Week	Learn	Key Resources	Build	Definition of Done
1	Docker & Compose basics; container networking; volumes; Git hygiene	- Docker Get Started - Docker Compose docs - Pro Git ch. 1-3	docker-compose.yml for Postgres + MinIO; repo skeleton, .gitignore, README stub; connect via psql and MinIO client	☐ docker compose up starts both services ☐ can query Postgres and browse MinIO console
2	REST APIs, rate limiting, exponential backoff, defensive JSON parsing, pytest mocking	- CoinGecko API docs - Real Python: requests - tenacity docs	extract_prices.py polling 15-20 coins on a schedule; retry/backoff on 429/5xx; structured JSON logging; pytest suite mocking failure cases	☐ runs unattended for 30+ min without crashing ☐ pytest suite green, covers a malformed-payload case

Phase gate — you're ready for Phase 2 when: the ingestion container can run continuously, log every failure it recovers from, and you can explain *why* a naive retry loop without backoff is dangerous against a rate-limited API.

Self-check: What happens to your pipeline if CoinGecko returns HTTP 200 with an empty body? What's the difference between a container volume and a bind mount, and why does it matter for Postgres data?

Phase 2 · Optimized Data Lake WEEKS 3-4				
Goal: move from raw JSON to a compressed, partitioned, query-efficient columnar layer.				
Week	Learn	Key Resources	Build	Definition of Done
3	Object storage concepts (buckets, keys); PyArrow tables; Parquet encoding & compression	- MinIO quickstart - Apache Arrow Python: Parquet	json_to_parquet.py converting raw payloads to Snappy-compressed Parquet in MinIO with Y/M/D/HH partitioning	☐ files browsable in MinIO console ☐ PyArrow round-trips data with correct dtypes
4	Querying Parquet directly with DuckDB; partition pruning; benchmarking methodology	- DuckDB: Why DuckDB - DuckDB https/S3 guide	DuckDB queries reading straight from the MinIO lake; a benchmark script comparing JSON vs. Parquet on disk size and scan time	☐ benchmark table committed to benchmarks/parquet_vs_json.md ☐ can state the compression ratio from memory

Phase gate: you can explain, with your own numbers, why columnar + partitioned beats row-based JSON for this workload — this becomes both an interview answer and your first thesis "experiment."

Self-check: Why does partitioning by hour help a query for "last 24 hours of BTC prices" but not one for "all-time average price of ETH"?

Phase 3 · Medallion Architecture with dbt WEEKS 5-7				
Goal: transform raw data into governed, tested, business-ready models. Heaviest phase — budget accordingly.				
Week	Learn	Key Resources	Build	Definition of Done
5	dbt core concepts: models, sources, ref() , materializations	- dbt Fundamentals course - dbt-duckdb adapter docs	dbt project on the DuckDB adapter; Bronze source views; Silver models (type casting, dedup, UTC normalization)	☐ dbt run succeeds end to end ☐ Silver row counts reconcile with Bronze
6	Window functions (SMA), joins across sources, SCD Type 2 via dbt snapshots	- dbt snapshots docs - SQL window functions tutorial	Gold models: fct_price_snapshot , dim_coin (snapshot), fct_volatility_alerts , fct_technical_indicators	☐ gold tables match a hand-calculated spot check ☐ dim_coin correctly versions a changed attribute
7	dbt testing (schema + singular tests), source freshness, documentation	- dbt tests docs - dbt-utils package	Tests across all layers (not_null, unique, accepted_values, a custom "no negative price" test); freshness checks; dbt docs generate	☐ dbt test fully green ☐ lineage graph screenshot saved to docs/

Phase gate: you can open the dbt docs lineage graph and narrate, column by column, how a Silver field became a Gold metric — this is the core "analytics engineering" competency employers screen for.

Self-check: Why put a "no negative price" test in dbt instead of just trusting the source? What's the difference between a dbt test and a Python unit test, in terms of what each one protects?

Phase 4 · Automated Orchestration WEEKS 8-9

Goal: replace manual runs with a scheduled, monitored, self-healing pipeline.

Week	Learn	Key Resources	Build	Definition of Done
8	DAG concepts, task dependencies, scheduling	Prefect tutorial or Airflow "Getting started"	A flow/DAG chaining: extract → land to lake → dbt run → dbt test, scheduled inside Compose	☐ flow completes end-to-end on a schedule, unattended
9	Task-level retries/backoff, notifications, backfills	- Prefect/Airflow notifications docs - Slack incoming webhooks	Simulated failure (e.g. kill the API mid-run) with automatic retry; Slack/email alert on repeated failure; a manual backfill flow for a historical date range	☐ screenshot/recording of a failure → alert → recovery cycle ☐ backfill flow re-processes a past day correctly

Phase gate: you can kill a task mid-pipeline and, without touching anything, watch it retry and either recover or alert you. This single demo is one of the strongest things you can show in an interview.

Self-check: What's the difference between retrying a task and re-running the whole DAG? Why might blind retries make a rate-limit problem worse instead of better?

Phase 5 · Quality, BI & Documentation WEEKS 10-11

Goal: prove correctness and package the project for reviewers — recruiters, professors, or both.

Week	Learn	Key Resources	Build	Definition of Done
10	Metabase basics; dashboard design principles (signal density, avoiding chart-junk)	- Metabase getting started - Dashboard design guide	Metabase connected to the Gold layer; 3–4 dashboards: live price board, volatility alerts, SMA/signal chart, cap segmentation	☐ dashboards live and screenshotted ☐ each chart answers one clear question
11	Technical writing; architecture diagramming; recording a concise demo	"Make a README" guide - diagrams.net	Final README with architecture diagram, setup instructions, design-decisions section; 2–3 min demo video; tagged v1.0 release	☐ a friend can go from <code>git clone</code> to a live dashboard in <10 min using only the README

Phase gate — project complete when: the repo is public, the README stands alone without you explaining it verbally, and the demo video shows a real failure-recovery moment, not just the happy path.

6 · Learning Resource Library

All free or free-tier. Search the exact title if a URL has moved — these are stable, official sources.

Topic	Resource	Type
Docker & Compose	Docker official "Get Started" guide (docs.docker.com/get-started); freeCodeCamp Docker & Compose crash course (YouTube)	Docs + video
Git	<i>Pro Git</i> book, chapters 1–3 (git-scm.com/book) — free	Book
Python HTTP/APIs	Real Python — "Python's Requests Library"; <code>tenacity</code> library docs for retry/backoff	Article + docs
CoinGecko API	docs.coingecko.com — free Demo plan: 100 calls/min, 10,000 calls/month, no credit card required	Official docs
Binance API (optional)	developers.binance.com — verify current regional availability before use	Official docs
MinIO	min.io/docs — Quickstart + Python SDK guide	Docs
PyArrow / Parquet	arrow.apache.org/docs/python — "Reading/Writing Parquet"	Docs
PostgreSQL	postgresql.org/docs — official tutorial; pgexercises.com for SQL practice	Docs + exercises
DuckDB	duckdb.org/docs — "Why DuckDB", Python API, https/S3 guide	Docs
dbt	learn.getdbt.com/courses/dbt-fundamentals (free course); docs.getdbt.com; dbt-duckdb adapter docs	Free course + docs
Orchestration	docs.prefect.io tutorial (recommended for solo projects — lighter weight); or airflow.apache.org "Getting started"	Docs
Metabase	metabase.com/docs — Getting Started guide	Docs
SQL refresher	sqlzoo.net or pgexercises.com if joins/window functions feel rusty	Interactive
Documentation craft	"Make a README" (makeareadme.com); diagrams.net for architecture diagrams	Guide + tool

7 · Skills-to-Job-Posting Alignment

Common Job Requirement	Where This Project Proves It
"Experience with containerized data environments"	Full Docker Compose stack, one-command bring-up
"Build resilient ingestion pipelines against third-party APIs"	Retry/backoff logic, rate-limit handling, mocked-failure unit tests
"Experience with cloud/object storage and columnar formats"	MinIO + partitioned Parquet lake, benchmarked vs. JSON
"SQL-based transformation with testing and documentation"	dbt Bronze/Silver/Gold models, tests, auto-generated docs
"Orchestration and monitoring of production pipelines"	Prefect/Airflow DAG with retries and failure alerting
"Communicate insights to non-technical stakeholders"	Metabase dashboards + recorded walkthrough

8 · Risks & Mitigations

Risk	Level	Mitigation
Falling behind the 5–8 hrs/week budget	High	Protect each phase's Definition of Done over extra polish; a working MVP beats a partial ideal build.
API rate limits or data-quality gaps (missing fields, inconsistent timestamps)	Medium	CoinGecko's free Demo plan comfortably covers 15–20 coins polled every few minutes; add schema validation at ingestion and dbt tests before Gold.
Scope creep across 5 phases	Medium	Each week ends with a checkbox Definition of Done — if it's not checked, don't start the next week's learning material.
Environment drift between local and container runs	Low	All services and connection strings defined in Docker Compose + <code>.env.example</code> .

9 · Bachelor Capstone Appendix (Academic Track)

This section reframes the exact same repository as a formal Computer Engineering capstone. Nothing here requires extra engineering work beyond what's already in the plan above — it's a different lens on the same artifact, so you can present it to recruiters as a portfolio project *and* submit it academically without redoing the build.

9.1 Problem Statement

Cryptocurrency markets generate high-frequency, high-volatility data that is difficult to monitor manually. This project investigates how a Modern Data Stack architecture — combining object storage, columnar file formats, SQL-based transformation, and workflow orchestration — can be applied to build a reliable, testable, and observable pipeline for real-time market monitoring, using cryptocurrency markets as a representative high-velocity use case.

9.2 Research Questions

RQ1 (Storage): How does converting raw JSON API responses to partitioned, compressed Parquet affect storage footprint and analytical query latency, compared to querying raw JSON directly?

RQ2 (Reliability): To what extent does automated testing (dbt) and orchestrated retry logic reduce data-quality failures and pipeline downtime, compared to an unmonitored script-based pipeline?

RQ3 (Alerting, optional/stretch): What threshold and windowing choices for volatility detection best balance alert responsiveness against false-positive rate on real market data?

9.3 Related Work

A short literature/technology review should cover: (1) the Lakehouse / Medallion architecture pattern popularized by Databricks and its predecessors in traditional data warehousing; (2) the "Modern Data Stack" movement and the ELT-over-ETL shift enabled by tools like dbt; (3) prior academic and industry work on financial market anomaly/volatility detection; (4) data quality and testing frameworks in data engineering. These are readily searchable via Google Scholar and each tool's own engineering blog (Databricks, dbt Labs, Fivetran) for grounding.

9.4 Methodology

An iterative, design-science approach: each of the five build phases functions as a design-build-evaluate cycle, with the Definition of Done per phase serving as the acceptance criterion. This maps naturally onto a thesis methodology chapter without alteration.

9.5 Evaluation Plan (Experiments to Run and Report)

Experiment	Metric(s)	Maps to
JSON vs. Parquet storage & query benchmark	Compression ratio; DuckDB scan latency at increasing data volumes	Phase 2 / RQ1
Pipeline fault injection	Recovery time; % of injected failures auto-recovered vs. requiring manual intervention	Phase 4 / RQ2
Data quality over time	% of dbt tests passing per run across the build period	Phase 3 / RQ2
Alert threshold tuning	Precision/recall of >5% alerts against a manually reviewed sample window	Phase 3 / RQ3

9.6 Limitations & Future Work

- Single-node local deployment rather than a distributed cloud environment (explicitly out of scope; note as future work: migrate to a managed warehouse/cloud object storage).
- Micro-batch polling rather than true event streaming (future work: Kafka/Kinesis-based ingestion for sub-second latency).
- Free-tier API constraints cap the number of tracked assets and polling frequency.

9.7 Thesis Chapter Mapping

Thesis Chapter	Draws From
1. Introduction & Problem Statement	Section 9.1, Section 1 (Executive Summary)
2. Related Work	Section 9.3
3. System Design & Architecture	Sections 2, 3, 4
4. Implementation	Section 5 (Phases 1–5), your commit history
5. Evaluation & Results	Section 9.5 experiment results, benchmark files in <code>benchmarks/</code>
6. Discussion, Limitations & Future Work	Section 9.6
7. Conclusion	Synthesis of RQ1–RQ3 findings

Tip: start a `docs/lab-notes.md` file in week 1 and jot down 3–5 lines after every session — decisions made, problems hit, numbers observed. This becomes the raw material for Chapters 4 and 5 and saves you from reconstructing the story from memory in week 11.

10 · Overall Success Metrics & Final Checklist

- ☐ Fresh clone reaches a populated Metabase dashboard in under 10 minutes via `docker compose up`
- ☐ 100% of dbt tests passing on the final build
- ☐ Orchestrated DAG demonstrably recovers from at least one injected failure
- ☐ Documented Parquet compression ratio vs. raw JSON
- ☐ Public GitHub repo, README, architecture diagram, and demo video published
- ☐ `docs/lab-notes.md` populated throughout — ready to become thesis Chapters 4–5